

Technical Implementation Plan: Tokyo Mate 東京通 | Photo Translate 拍照翻譯

Branch: `002-photo-translate` | **Date:** 2026-09-19 | **Spec:** `spec.md`

Input: Feature specification from `specs/002-photo-translate/spec.md` · UX/UI Design Handoff `specs/002-photo-translate/ux-ui-design-handoff.md`。

Technical Truth & Plan 邊界說明

重要架構原則 (沿用 001 慣例) :

1. **Specification 是 Product Truth** : 定義本功能的核心承諾、FR、SC 與隱私邊界。
2. **Plan 是 Technical Truth** : 本文件內之技術選型是實現 Specification 的機制，不是產品契約本身。
3. **變更邊界** : 技術實作改變但產品行為不變時，僅需更新本 Plan；若產品行為、Scope、隱私邊界或核心 AI 體驗改變，MUST 先更新 Specification 並重新批准。

Summary

讓使用者從既有首頁進入「拍照翻譯」，以既有 file input / camera capture 取得照片、以新增的 touch-friendly 選取框元件框選文字區域、經新增的 `/api/photo-ocr` serverless function 呼叫既有 OpenAI Responses API (vision-capable model) 取得 OCR 原文，再經新增的 `/api/photo-translate` serverless function 以既有 OpenAI Responses API 將 OCR 原文譯為使用者選擇的繁體中文或日本語，最後 reuse 既有 `/api/speech` 播放對應語音。全流程沿用既有 React + TypeScript + Vite + Vercel Serverless Functions 架構與 server-side secret boundary，不新增 provider、不新增 frontend dependency、不新增資料庫或全域狀態庫。

Technical Context

Language/Version: TypeScript 5.x (沿用既有 tsconfig)

Primary Dependencies: React 18、Vite、`openai` SDK (既有) ；不新增 frontend/runtime dependency

Storage: 無 (runtime/session memory only，不寫入 LocalStorage/IndexedDB/資料庫)

Testing: Vitest + React Testing Library + jsdom (沿用既有 `tests/api/*`、`src/**/*test.tsx` 慣例)

Target Platform: 既有 Vercel 部署 (Web / mobile-first installable PWA，PWA 架構本身不擴張)

Project Type: 既有單一 Repository (Vite frontend + Vercel serverless `api/`) 之 incremental feature

Performance Goals: 沿用既有 client-to-render 量測精神；OCR/翻譯呼叫各自獨立可重試，不阻塞既有 MVP 路徑

Constraints: 不得新增 dependency 而未經批准；不得新增第二 OCR/翻譯 provider；不得建立永久照片/OCR/翻譯歷史；secrets 僅存在 server-side

Scale/Scope: 單一新功能畫面 (Photo Translate) + 兩個新 serverless endpoints，不影響既有 001 路由與資料流

Constitution Check

GATE: Must pass before Phase 0 research. Re-check after Phase 1 design.

- **I. Specification Before Implementation**：本 Plan 對齊已批准 `spec.md` 與 `ux-ui-design-handoff.md`；Sync Check 已確認「未經確認選取區域不得整張 OCR」語意已存在於 Specification，未修改 Specification。
- **II. Simplest Sufficient Technology**：Region selection 採自建輕量元件（見 Research 決策），不引入 crop library；OCR / 翻譯 reuse 既有 `openai` SDK 與既有 Responses API 呼叫模式；Speech 完全 reuse 既有 `/api/speech`，無程式碼變更。
- **III. Complexity Discipline**：複雜度分類維持 C | AI-Powered App；本功能未新增 Database、Auth、微服務或全域狀態層。
- **IV. Technology Stack Stability**：沿用既有 React + TS + Vite + Vercel Functions + OpenAI；無 provider 或 framework 替換。
- **V. Privacy by Minimum Necessary Data**：照片、選取區域、OCR 原文與翻譯僅存在 runtime state；離開功能或更換照片立即清除（依 FR-014 / FR-016）。
- **VI. Secrets Never Live in Frontend**：`OPENAI_API_KEY` 僅存在既有 `api/_lib/openai.ts` server-side boundary，新 endpoints 沿用同一 helper，不在前端新增任何金鑰。
- **VII. AI Freshness and Honesty**：OCR / 翻譯無可靠結果時不得虛構內容，依 FR-007 / FR-009 / FR-017 呈現誠實狀態。
- **VIII. Translation Quality Over Literal Translation**：翻譯 prompt 沿用既有繁中 / 日文自然語氣要求，日文輸出避免不自然逐字翻譯，繁中輸出避免大陸用語。
- **IX. Safety-Critical Action First**：本功能非安全關鍵情境，不影響既有 Emergency 優先順序。
- **X. Testability**：Phase 1 定義 contracts 與 quickstart 驗證情境，涵蓋成功 / 失敗 / stale-state 邊界。
- **XI. Maintainability**：新增程式碼依既有 `api/`、`src/features/`、`src/screens/`、`src/types/`、`src/services/` 慣例分層，不建立巨型檔案。
- **XII. Incremental Implementation**：後續 `/speckit.tasks` 應維持 phase-by-phase (Photo Acquisition → Region Selection → OCR → Translation → Speech → Lifecycle)。
- **XIII. Graceful Failure**：所有新 endpoint 沿用既有 `success()/failure()` envelope 與一般化 `ProductError`，不暴露 provider 錯誤。
- **XIV. Repository Safety**：僅新增檔案，不搬移或覆寫既有 `specs/001-tokyo-travel-assistant/`、`.github/`、`.specify/` 等資產。
- **XV. Traditional Chinese Working Language**：本 Plan 與後續產出 artifacts 均使用台灣繁體中文。

無違反項目，**Complexity Tracking** 表格從略。

Project Structure

Documentation (this feature)

```

specs/002-photo-translate/
├── spec.md                # Product Truth ( 已批准 )
├── ux-ui-design-handoff.md # UX/UI Handoff ( 已批准 )
├── plan.md                # 本檔案
├── research.md            # Phase 0 產出
├── data-model.md          # Phase 1 產出
├── quickstart.md          # Phase 1 產出
├── contracts/
│   ├── photo-ocr-api.md
│   └── photo-translate-api.md
└── checklists/

```

Source Code (repository root)

```

tokyo-mate/
├── api/
│   ├── photo-ocr.ts        # [新增] 選取區域 OCR：影像 → 可靠原文
│   ├── photo-translate.ts  # [新增] OCR 原文 → 繁體中文 / 日本語翻譯
│   ├── speech.ts          # [沿用，不修改] 既有 zh-TW/ja 語音合成
│   └── _lib/
│       └── openai.ts       # [沿用，不修改] server-side OpenAI
├── client/model
│   └── http.ts             # [沿用，不修改] success/failure envelope
├── helpers
│   └── prompts/
│       ├── photo-ocr.ts    # [新增] OCR 指令樣板
│       └── photo-translate.ts # [新增] 翻譯指令樣板
├── src/
│   ├── screens/
│   │   └── PhotoTranslateScreen.tsx # [新增] 狀態容器，串接各 phase
│   ├── features/
│   │   └── photo-translate/
│   │       ├── PhotoAcquisitionPanel.tsx
│   │       ├── RegionSelector.tsx      # 自建、無新依賴之選取框元件
│   │       ├── OcrResultPanel.tsx
│   │       └── TranslationPanel.tsx
│   ├── types/
│   │   └── photoTranslate.ts           # [新增] Runtime task/state 型別
│   ├── services/
│   │   └── api.ts                     # [擴充] 新增 requestPhotoOcr /
├── requestPhotoTranslate
├── tests/
│   └── api/
│       ├── photo-ocr.test.ts          # [新增]
│       └── photo-translate.test.ts    # [新增]

```

Structure Decision：延續既有「**api/**」 endpoint 一檔案、**src/features/<domain>/** 放 UI、**src/screens/** 放狀態容器、**src/types/** 放共用型別、**src/services/api.ts** 集中 fetch wrapper」慣

例，僅新增檔案，不調整既有目錄結構、不新增 routing library (**HomeScreen** 仍以既有條件式 **render** 模式切換畫面) 。

Complexity Tracking

本 *Plan* 無 *Constitution Check* 違反項目，故本表從略。